

# ■第七课：变量进阶与**LED**点阵

7.1 变量的作用域

7.2 变量的存储类别

7.3 点阵的初步认识

7.4 点阵的图形显示

7.5 点阵的动画显示

# 7.1 变量的作用域

## ■ 1、局部变量

函数内部声明的变量，只在本函数内有效，在本函数以外是不能使用的，叫做局部变量。

## ■ 2、全局变量

在函数外部声明的变量就是全局变量。一个源程序文件可以包含一个或者多个函数，全局变量的作用范围是从它开始声明的位置一直到程序结束。

全局变量定义放在最开始，方便查找、维护、修改。

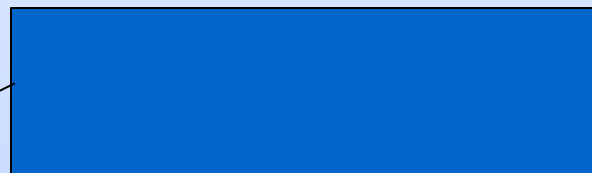
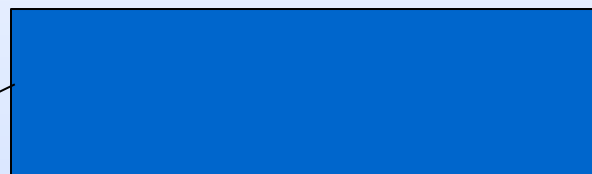
# 7.1 变量的作用域

头部

变量声明

主函数

中断函数



仅中断函数用到全局变量，可以放在中断函数之前吗？

按照规范，局部变量定义在函数内部，全局变量声明都定义在最开始，方便统一查找、维护、修改、升级。

全局变量用起来比较方便，可直接调用，为什么要有一些局部变量呢？

## 全局变量的负作用

- 1、降低函数的**独立性**，任何一个函数的对其修改都可能影响到其他函数。
- 2、降低函数的**通用性**，不利于函数的重复调用（此次调用实现，下次调用全局变量改变返回值可能不同）。
- 3、降低程序的**清晰度**，每个函数执行都有可能改变全局变量值，无法清楚判断每个时刻的全局变量的值。
- 4、全局变量**永久占据**内存单元。

原则：能用局部变量就不用全局变量

全局变量和局部变量尽量不同名，如果同名，在局部变量作用域范围内，局部变量有效。

## 7.2 变量的存储类别

### ■ 1、自动变量

函数中的局部变量，如果不加**static**关键字修饰，都属于自动变量，也叫做动态变量（**auto**省略）。

### ■ 2、静态变量

**所有的全局变量都属于静态变量**，局部变量如果加了**static**关键字修饰的话，也是静态变量。

举例：

静态变量 类似 买房子，永久使用

自动变量 类似 租房子 进入函数就使用 出函数 就会释放RAM地址

比如在中断函数中定义局部静态变量：

`static unsigned char i=0`---只初始化一次

`unsigned char i=0` ---每次进入都初始化为0

则：`i`和`cnt`都从全局变量，进入中断函数，变成局部变量，并保持为静态变量。



```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 unsigned char LedChar[16]={
10     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
11     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
12 };
13 unsigned char LedBuff[6]={    //数组显示数码管值，经常变化，不能在FLASH中，应该在RAM
14     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF
15 };
16 unsigned char flag1s = 0;
17 unsigned int cnt = 0; //char最大255, 1ms*1000=1s, 最大要1000, 因此定义int型
18 unsigned char i = 0; //定义第几个数码管
19
20 void main() {
21
22     unsigned long sec = 0;
23     EA = 1;
24     ET0 = 1;
```

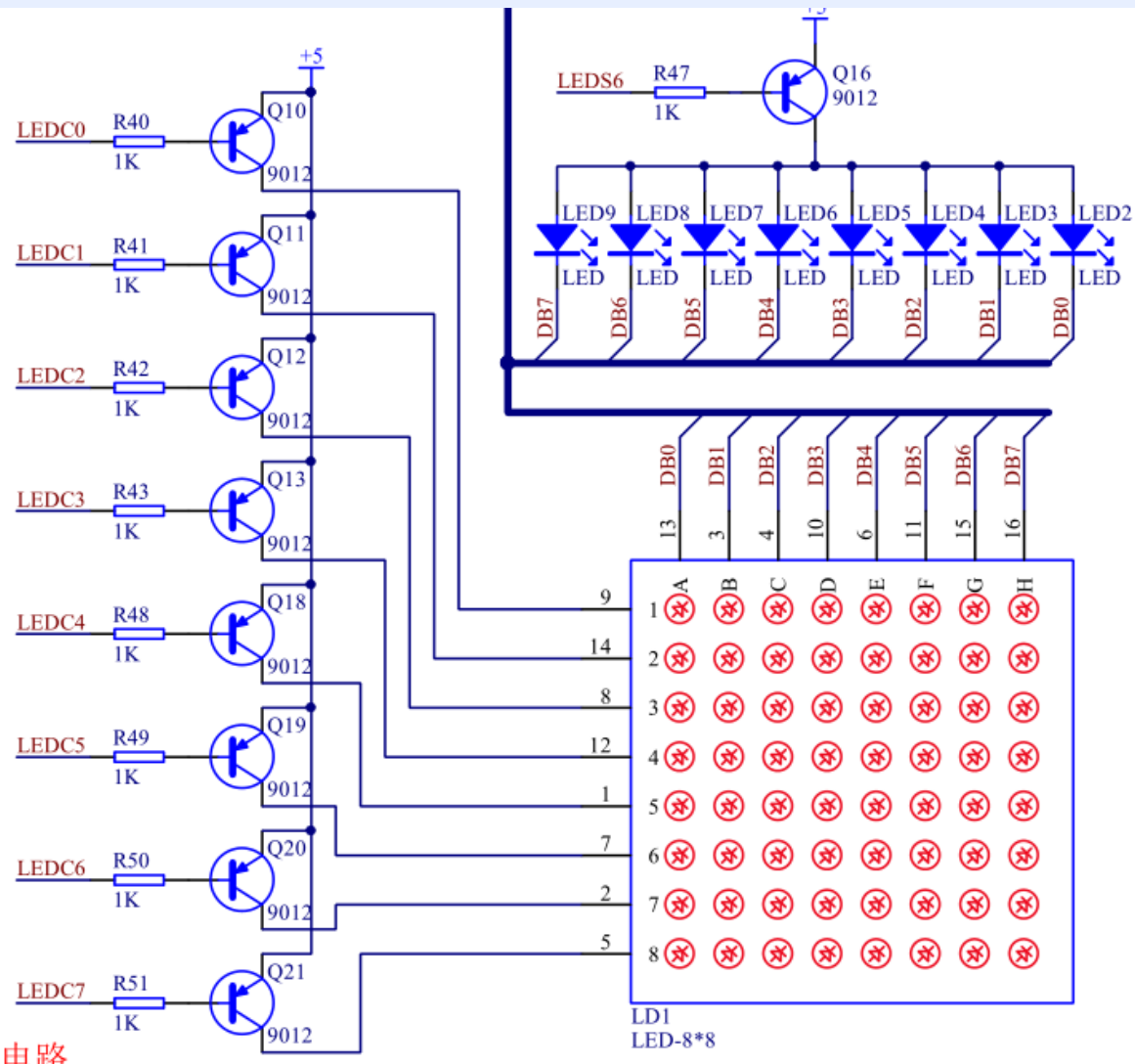
```
48 void InterruptTime0() interrupt 1 //中断函数后面有关键字和数字；不需要调用，达到条件，自动进入
49 {
50     static unsigned int cnt = 0; //char最大255, 1ms*1000=1s, 最大要1000, 因此定义int型
51     static unsigned char i = 0; //定义第几个数码管
52
53     TH0 = 0xFC; //进入定时器中断硬件自动清零
54     TL0 = 0x67;
55     cnt++;
56     if(cnt >= 1000)
57     {
58         cnt = 0;
59         flag1s = 1;
60     }
61
62     P0 = 0xFF;
63     switch(i)
```

## 7.3 点阵LED初步认识

- 点阵**led**灯是整齐排列的，被镶嵌到一块塑料板上，可以显示汉字和图形，用来介绍商品，吸引顾客。
- 点阵**led**显示屏作为一种现代电子媒体，具有灵活的显示面积（可分割、任意拼装）、高亮度、长寿命、大容量、数字化、实时性的特点。



# 点阵LED



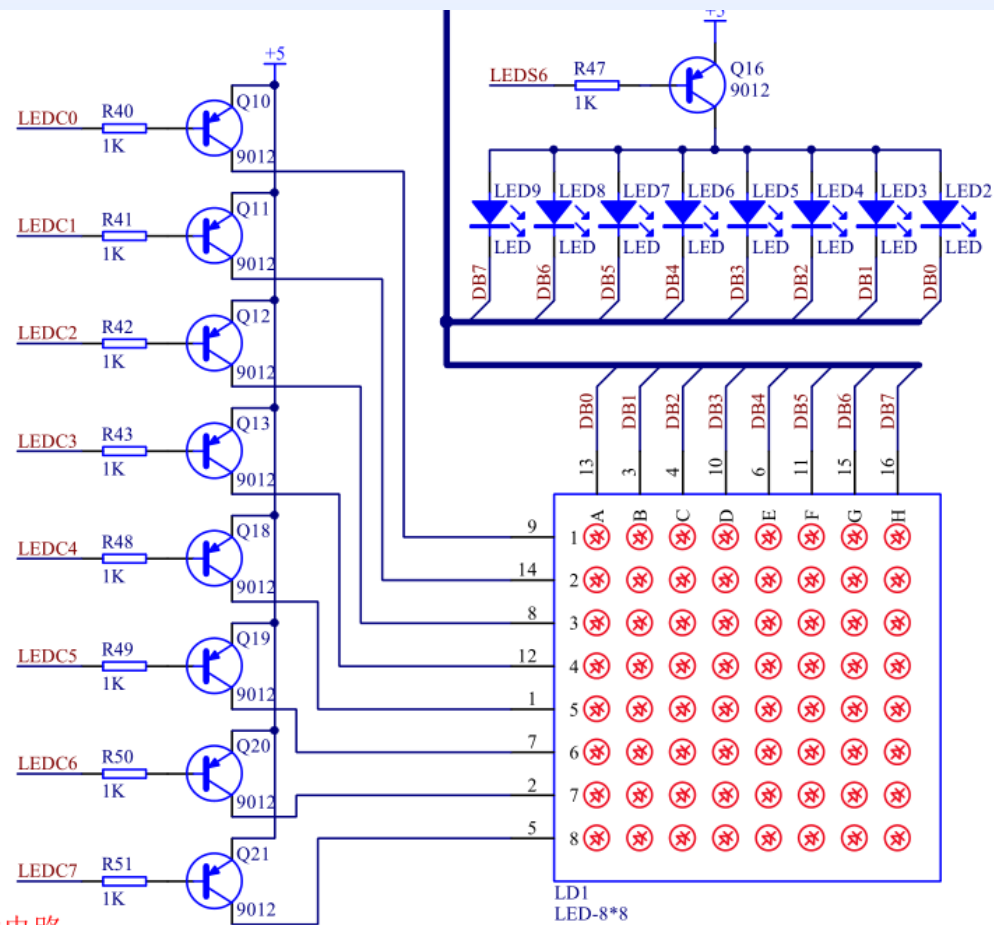
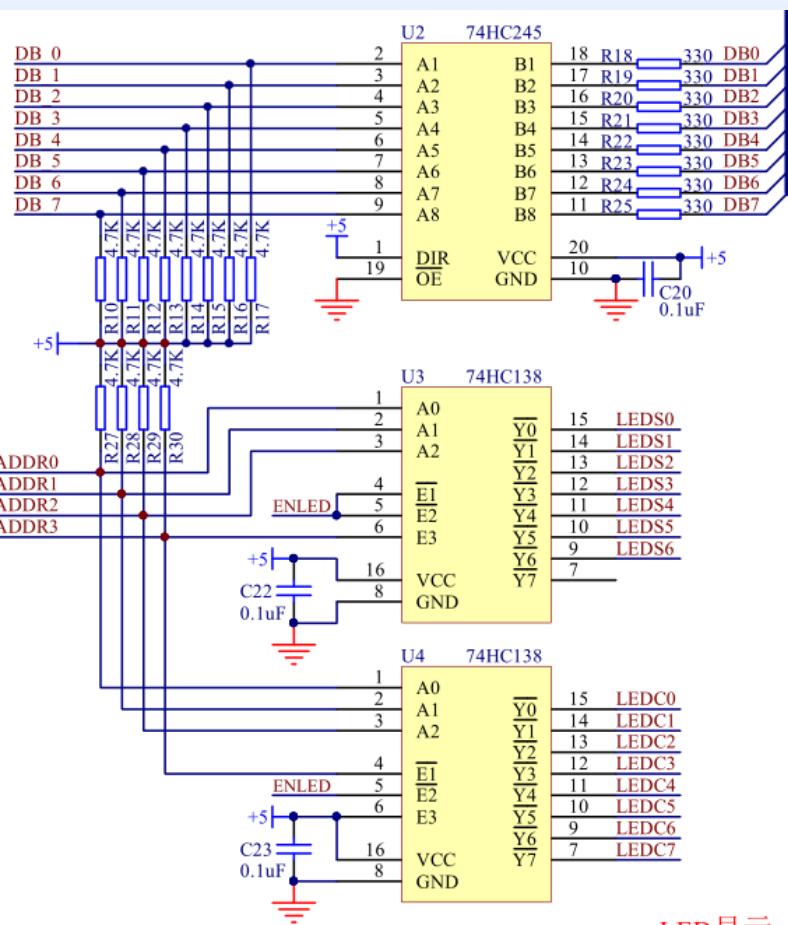
# 点阵LED原理

- 64个小灯，可以点亮任意的小灯
- 任意拼装组合8\*8 16\*16 16\*32

如：DC1高电平 DR8低电平 右下角的小灯点亮



# 点阵LED



LED显示、及其驱动电路

# 点亮点阵（某点/某行）

```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8 sbit LED = P0^0;
9
10 void main()
11 {
12     ENLED = 0; //U4工作
13     ADDR3 = 0;
14     ADDR2 = 0;
15     ADDR1 = 0;
16     ADDR0 = 0;
17
18     LED = 0;
19     while(1);
20 }
```

```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9
10 void main()
11 {
12     ENLED = 0; //U4工作
13     ADDR3 = 0;
14     ADDR2 = 0;
15     ADDR1 = 0;
16     ADDR0 = 0;
17
18     P0 = 0x00;|
19     while(1);
20 }
```

左边：左上角小灯点亮

右边：第一行全部点亮

要让点阵8行都显示，也需要动态显示。

# 点亮点阵（全部）

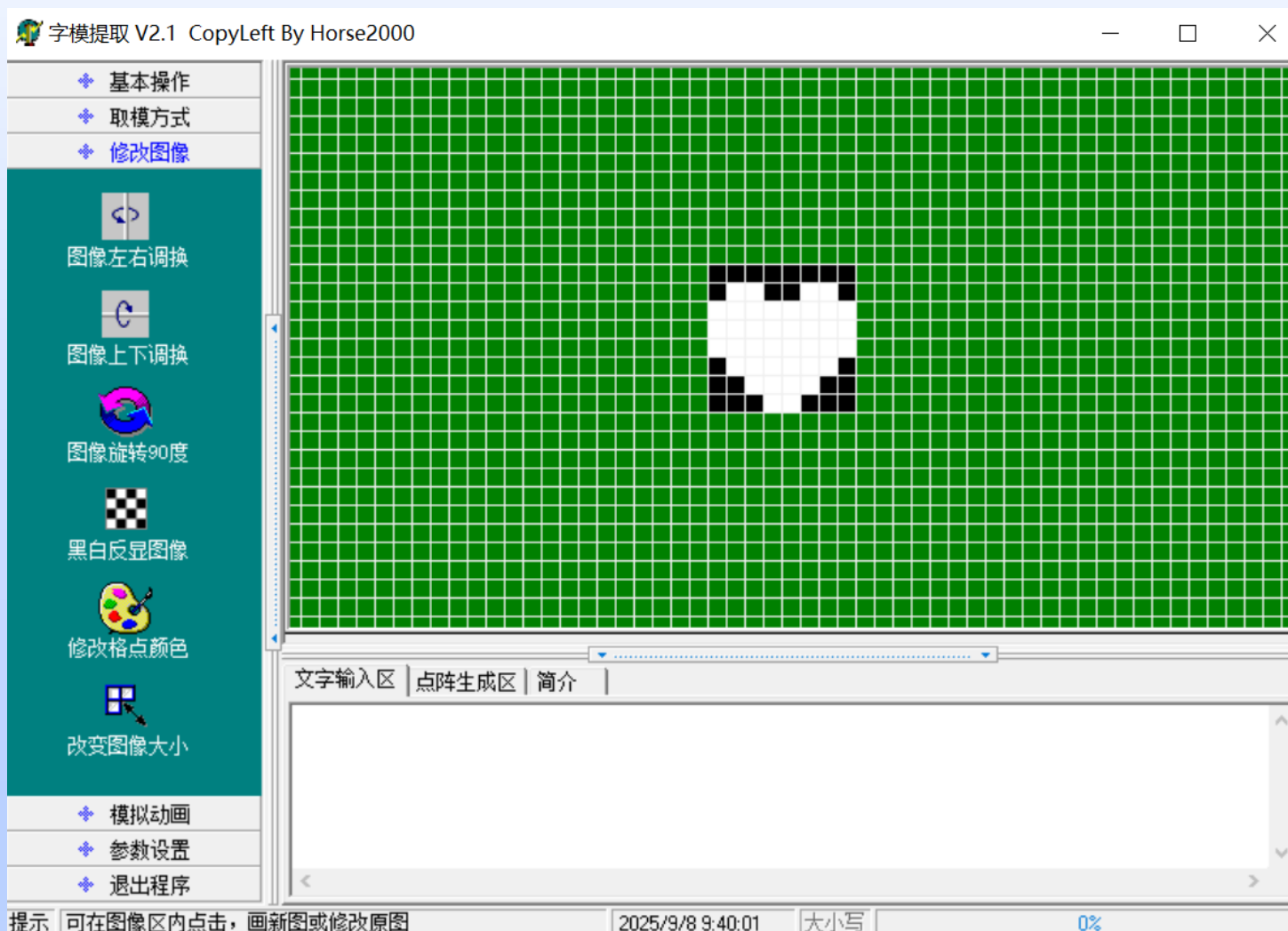
```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9
10 void main()
11 {
12     EA = 1; //开启总中断
13
14     ENLED = 0; //U4工作点阵的138芯片工
15     ADDR3 = 0;
16
17     TMOD = 0x01;
18     TH0 = 0xFC;
19     TL0 = 0x67;
20     ET0 = 1; //打开定时器中断
21     TR0 = 1; //打开定时器
22     while(1);
23 }
```

```
void InterruptTimer0() interrupt 1
{
    static unsigned char i = 0; //刷新点阵某行

    TH0 = 0xFC;
    TL0 = 0x67;

    P0 = 0xFF; //消隐
    switch(i)
    {
        case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; P0=0x00; break;
        case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; P0=0x00; break;
        case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; P0=0x00; break;
        case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; P0=0x00; break;
        case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; P0=0x00; break;
        case 5: ADDR2=1; ADDR1=0; ADDR0=1; i++; P0=0x00; break;
        case 6: ADDR2=1; ADDR1=1; ADDR0=0; i++; P0=0x00; break;
        case 7: ADDR2=1; ADDR1=1; ADDR0=1; i=0; P0=0x00; break;
        default: break;
    }
}
```

## 7.4 点阵的静态图形显示



使用：新建图象-模拟动画（放大格点，方便观察）-鼠标点击“心形”出来-修改图形（黑白反显图像）-参数设置（其他选项）-取模方式

C51:0xFF,0x99,0x00,0x00,0x00,0x81,0xC3,0xE7,



## 7.4 点阵的静态图形显示

选项

取模方式

☒ 横向取模

☐ 纵向取模

1. 当您输入完文字后, 可以按Ctrl+Enter表示您想结束文字的输入, 但由于要对每个文字单独取模, 在把文字转化为图像的处理方法上有所不同, 具体说来是如果您要横向取模的话则把输入的文字纵向排列成图像而纵向取模的话则把输入的文字横向排列成图像;

☒ 字节倒序

字节倒序是为了满足某些液晶的要求而设。即一个字节倒过来, 比如 0x17, 要把它变成0xe8。

保留文字字模数据的最后一个逗号

☒ 保留

如果保留的话, 如下所示:

```
/*— 文字: 中 —*/  
/*— 宋体12: 此字体下对应的点阵为: 宽x高=16x16 —*/  
0x01, 0x00, 0x01, 0x00, 0x21, 0x08, 0x3F, 0xFC, 0x21, 0x08, 0x21, 0x08, 0x21, 0x08, 0x21, 0x08,
```

关于数据压缩的一点说明

压缩的功能只对图像有效。对单个文字不建议这么做, 因为单个文字本身占的字节数较少, 压缩率不高或反而增加字节数。对大图像而言, 其中连续的 00, FF较多, 故压缩显得有必要。现提供的压缩方法如下 (作者注: 该方法由龙啸九天提出): 2个以上 (包含2个但不超过 255个) 连续的00或FF字节可仅用2个连续字节代替, 第一个字节放00或FF, 紧跟的第二个字节放连续的00或FF的个数。用者请在自己的程序中自行解码。所以, 在每遇到单个的00或FF时, 会导致增加1个字节数; 在遇到连续2个00或FF时, 字节数不变; 在遇到3个以上时则会比原来的

A51数据的位数

☐ 只在字母前加0

☒ 任何时候都加0

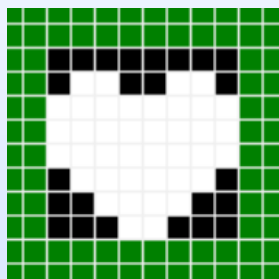
主要是为了当生成 A51 格式数据时, 若为 AA 之类的数前边应加 0, 这就出现有的是2位, 有的是3位的形式显示数据, 而有时候也为了美观的缘故都加 0, 以3位数的形式显示。

确定 取消

横向取模: P0:控制横向。

字节倒序: 通常 (数字左侧高位右侧低位), 图中是低位在左侧, 是倒序。

# 点阵的静态图形显示



0xFF,0x99,0x00,0x00,  
0x00,0x81,0xC3,0xE7

左侧低位右侧高位

简单图形8\*8即可，显示汉字一般需要16\*16。



```

1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 unsigned char code image[] = {
10 0xFF,0x99,0x00,0x00,0x00,0x81,0xC3,0xE7
11 };//数组存放心形字符
12 void main()
13 {
14     EA = 1;//开启总中断
15
16     ENLED = 0; //U4工作点阵的138芯片工作
17     ADDR3 = 0;
18
19     TMOD = 0x01;
20     TH0 = 0xFC;
21     TL0 = 0x67;
22     ET0 = 1;//打开定时器中断
23     TR0 = 1;//打开定时器
24     while(1);
25 }

```

```

26
27 void InterruptTimer0() interrupt 1
28 {
29     static unsigned char i = 0;//刷新点阵某行
30
31     TH0 = 0xFC;
32     TL0 = 0x67;
33
34     P0 = 0xFF;//消隐
35     switch(i)
36     {
37         case 0:ADDR2=0;ADDR1=0;ADDR0=0;i++;P0=image[0];break;
38         case 1:ADDR2=0;ADDR1=0;ADDR0=1;i++;P0=image[1];break;
39         case 2:ADDR2=0;ADDR1=1;ADDR0=0;i++;P0=image[2];break;
40         case 3:ADDR2=0;ADDR1=1;ADDR0=1;i++;P0=image[3];break;
41         case 4:ADDR2=1;ADDR1=0;ADDR0=0;i++;P0=image[4];break;
42         case 5:ADDR2=1;ADDR1=0;ADDR0=1;i++;P0=image[5];break;
43         case 6:ADDR2=1;ADDR1=1;ADDR0=0;i++;P0=image[6];break;
44         case 7:ADDR2=1;ADDR1=1;ADDR0=1;i=0;P0=image[7];break;
45         default:break;
46     }
47 }

```

## 7.5 点阵的动画显示

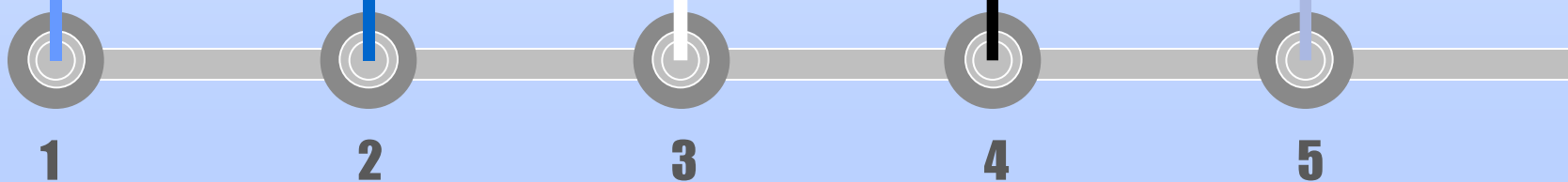
点阵取模将动画所需的所有图片都取出。I♥U，加上头尾。每张图8行8列，共40行。

数组更新为40

中断函数中静态变量：  
`index = 0;`  
`P0=image[index + 0]`

起始行; `tmr = 0;`//切换  
图片的时间 `if(tmr>=250)`  
`{tmr = 0;index++;}`

`if(index >= 32)//最多到  
33行{index = 0;}`



动画是什么：多张图片切换组合显示的动画，并非真正动。快速拍摄照片，照片快速播放。用程序切换多张图片。



40行的图片  
内部显示**33**张图片

第一张:  $8*8$  (1-8行)

第二张:  $8*8$  (2-9行)

...

第二张:  $8*8$  (33-40行)



40行的图片  
内部显示**33**张图片

第一张：8\*8（1-8行）

第二张：8\*8（2-9行）

...

第二张：8\*8（33-40行）

每当经过一段时间就刷新  
一张图片。

```

1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 unsigned char code image[] = {
10     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
11     0xC3, 0xE7, 0xE7, 0xE7, 0xE7, 0xE7, 0xC3, 0xFF,
12     0x99, 0x00, 0x00, 0x00, 0x81, 0xC3, 0xE7, 0xFF,
13     0x99, 0x99, 0x99, 0x99, 0x99, 0xC3, 0xC3, 0xFF,
14     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF
15 }; //数组存放心形字符
16 void main()
17 {
18     EA = 1; //开启总中断
19
20     ENLED = 0; //U4工作点阵的138芯片工作
21     ADDR3 = 0;
22
23     TMOD = 0x01;
24     TH0 = 0xFC;
25     TL0 = 0x67;
26     ET0 = 1; //打开定时器中断
27     TR0 = 1; //打开定时器
28     while(1);
29 }

```

```

30
31 void InterruptTimer0() interrupt 1
32 {
33     static unsigned char i = 0; //刷新点阵某行
34     static unsigned char index = 0; //起始行
35     static unsigned char tmr = 0; //切换图片的时间
36     TH0 = 0xFC;
37     TLO = 0x67;
38
39     PO = 0xFF; //消隐
40     switch(i)
41     {
42         case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; PO=image[index + 0]; break;
43         case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; PO=image[index + 1]; break;
44         case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; PO=image[index + 2]; break;
45         case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; PO=image[index + 3]; break;
46         case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; PO=image[index + 4]; break;
47         case 5: ADDR2=1; ADDR1=0; ADDR0=1; i++; PO=image[index + 5]; break;
48         case 6: ADDR2=1; ADDR1=1; ADDR0=0; i++; PO=image[index + 6]; break;
49         case 7: ADDR2=1; ADDR1=1; ADDR0=1; i=0; PO=image[index + 7]; break;
50         default: break;
51     }
52
53     tmr++;
54     if(tmr>=250)
55     {
56         tmr = 0;
57         index++; //每过250ms就向下移位
58         if(index >= 32) //最多到33行
59         {
60             index = 0;
61         }
62     }
63 }

```

# 点阵的动画显示-横向移动

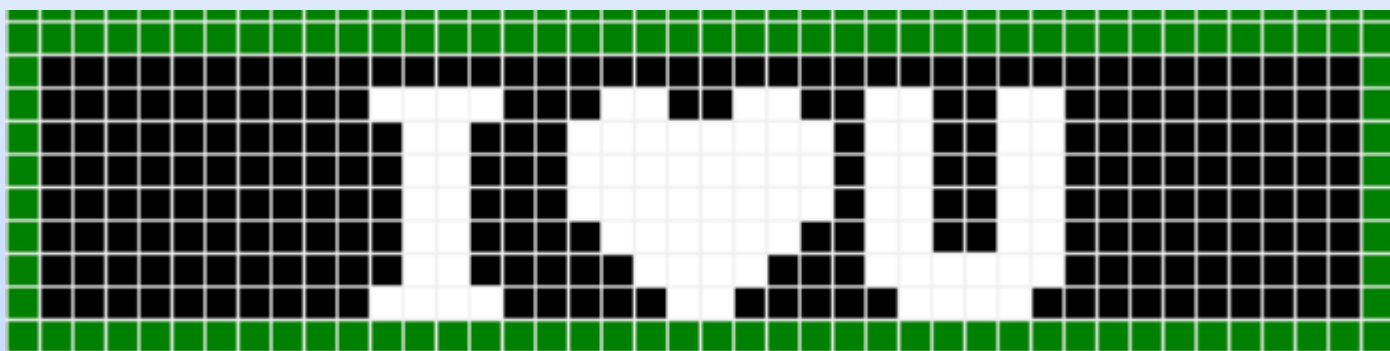
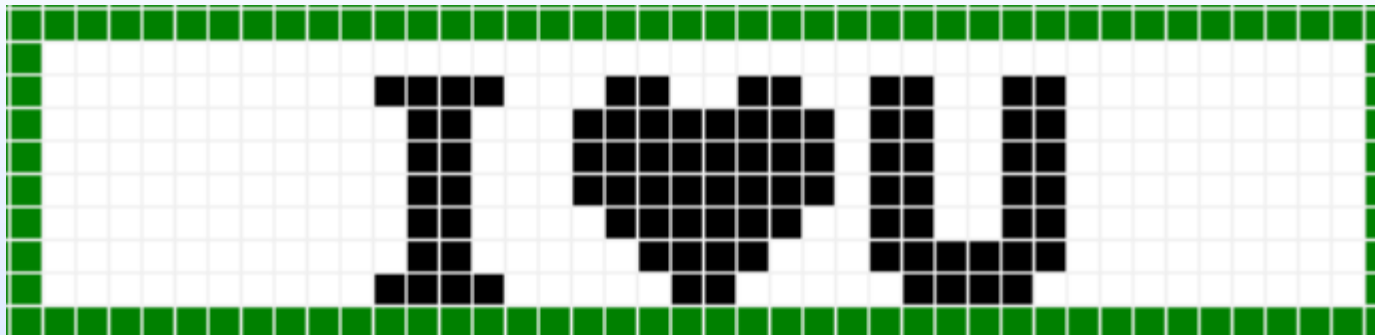
开发板的原理图设计-方便上下移动

要左右移动可以借助二位数组。

- 数组类型 数组名[数组长度1][数组长度2];
- 例如 **unsigned char a[2][3];**
- 定义一个**2行3列**的无符号字符型二维数组

|         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|
| a[0][0] | a[0][1] | a[0][2] | a[1][0] | a[1][1] | a[1][2] |
|---------|---------|---------|---------|---------|---------|

- **unsigned char a[2][3]={1,2,3, {4,5,6}};**
- **unsigned char a[2][3]={1,2,3,4,5,6};**
- **unsigned char a[2][3]={1,2},{3,4};**
- **unsigned char a[2][3]={1,2,0,3,4,0};**
- **unsigned char a[2][3]={1,2,3,4};**
- **unsigned char a[2][3]={1,2,3},{4,0,0};**



第一张：8\*8（1-8列）

第二张：8\*8（2-9列）

...

 706横向显示30张图片

30个动画：帧图片

中断函数：P0=image[index][0]

数组元素一行一行动态刷新



# 作业

## ■ 操作练习：

1、尝试用点阵做一个**9**到**0**的倒计时牌显示。